TokenZswap · Foundry 테스트 해설

테스트 코드 읽는 법

`code/test/AMM.t.sol` (풀 6개)과 `code/test/NFTMarket.t.sol` (NFT 마켓 8개), 테스트 14개를 한 줄씩 해설합니다. 각 테스트가 **무엇을 증명하는지**, 줄마다 **지갑과 풀이 어떻게 변하는지**, 실패하면 **어떻게 읽는지**를 다룹니다.

14	14	0
테스트	통과	실패

실행: `cd code && forge test -vv`

목차

1. 테스트란 무엇이고 왜 쓰나
2. Foundry 테스트 문법 한눈에 보기
3. AMM.t.sol: 풀 테스트 6개
4. NFTMarket.t.sol: NFT 마켓 테스트 8개
5. 실패 메시지 읽는 법
6. 서브젝트 요구사항과 테스트 대응표
7. 부록: 테스트 파일 전체

1. 테스트란 무엇이고 왜 쓰나

컴파일러는 **문법**만 검사합니다. **로직**이 맞는지는 실행해 봐야 압니다. 이 프로젝트에서 실제로 있었던 일 세 가지가 그 예입니다.

버그	컴파일	결과	잡은 것
`addLiquidity` 첫 공급을 `sqrt(reserveA * reserveB)` 로 씀	성공	빈 풀에서 0 × 0 = 0 → 첫 공급이 영원히 실패	첫 공급 테스트
`swap` 에서 `require(amountOut > 0)` 를 계산 **전에** 둠	성공	`amountOut` 이 아직 0이라 모든 swap이 revert	swap 테스트 2개가 실패
`cancelListing` 에서 `Listing memory` 사용	성공	취소가 장부에 저장되지 않음 → 옛 등록으로 NFT를 살 수 있음	취소 테스트 (일부러 바꿔서 실패 확인함)

Foundry 테스트는 **가짜 블록체인 위에서** 컨트랙트를 배포하고, 함수를 호출하고, 결과가 기대와 같은지 확인합니다. 테스트넷도, MetaMask도, 가스비도 필요 없습니다.

테스트를 쓰는 요령. 빈칸 앞에서 항상 이렇게 물어보세요. *"이 상황에서 이 값은 몇이 나와야 하지?"* 그 숫자를 손으로 계산해서 `assertEq` 의 오른쪽에 적으면 됩니다. 계산은 `documentation/amm-math.md` 의 공식표를 씁니다.

2. Foundry 테스트 문법 한눈에 보기

문법	뜻
`contract XTest is Test`	forge-std의 `Test` 를 상속해야 아래 기능을 쓸 수 있음
`setUp()`	**각 테스트 함수 실행 전마다** 새로 실행. 테스트끼리 상태가 섞이지 않음
`test_이름()`	이름이 `test` 로 시작하는 함수만 테스트로 실행
`100 ether`	ETH가 아니라 `100 × 1018` 이라는 숫자. 소수점 18자리 토큰 100개
`makeAddr("alice")`	테스트용 가짜 주소
`vm.prank(x)`	**바로 다음 호출 1번만** `msg.sender` 를 x로
`vm.startPrank(x)` ~ `vm.stopPrank()`	그 **사이의 모든 호출**을 x로
`assertEq(a, b)`	a == b. 왼쪽 = 실제 값, 오른쪽 = 기대 값
`assertGe / assertGt`	a ≥ b / a > b (Greater or equal / Greater than)
`assertLe / assertLt`	a ≤ b / a < b (Less or equal / Less than)
`assertTrue / assertFalse`	bool이 참 / 거짓
`assertXxx(a, b, "메시지")`	실패했을 때 이 메시지가 같이 출력됨. 어느 줄인지 바로 보임
`vm.expectRevert(bytes("msg"))`	**바로 다음 호출**이 이 메시지 **그대로** revert해야 통과
`vm.expectEmit(...)` + `emit X(...)`	**바로 다음 호출**이 이 이벤트를 내야 통과

vm은 어디서 오나. `vm` 은 주소 `0x7109…D12D` 를 `Vm` 인터페이스로 감싼 것입니다. 이 주소에는 컨트랙트가 없고, Foundry가 이 주소로 가는 호출을 가로채서 Rust 코드로 처리합니다. 그래서 진짜 체인에서는 불가능한 일(남의 `msg.sender` 로 행세하기)을 할 수 있고, 이걸 치트코드라고 부릅니다. 반면 `assertEq` 는 `StdAssertions.sol` 에 Solidity로 구현되어 있고, 비교가 실패할 때만 `vm.assertEq` 를 불러 실패를 보고합니다.

자주 쓰는 실행 옵션

명령	용도
`forge test -vv`	전체 실행, `console.log` 출력 보기
`forge test --match-test 이름 -vvvv`	테스트 하나만, 모든 호출과 반환값을 트레이스로 보기
`forge test --rerun`	방금 실패한 테스트만 다시

3. AMM.t.sol: 풀 테스트 6개

3-1. 준비: 상태 변수와 setUp

AMM.t.sol · 10–28

```
10  contract AMMTest is Test {
11      AMM amm;
12      SunWoo42 tokenA;
13      TokenB tokenB;
14
15      address alice = makeAddr("alice");
16      address bob = makeAddr("bob");
17
18      /// @dev 각 테스트 전에 실행: 토큰 2개 + AMM 배포, alice/bob에게 토큰 나눠주기
19      function setUp() public {
20          tokenA = new SunWoo42(1_000_000 ether); // 이 테스트 컨트랙트가 배포자 → 전량 받음
21          tokenB = new TokenB(1_000_000 ether);
22          amm = new AMM(address(tokenA), address(tokenB));
23
24          tokenA.transfer(alice, 10_000 ether);
25          tokenB.transfer(alice, 10_000 ether);
26          tokenA.transfer(bob, 10_000 ether);
27          tokenB.transfer(bob, 10_000 ether);
28      }
```

`new SunWoo42(...)` 로 배포하는 주체는 **테스트 컨트랙트 자신**입니다. SunWoo42의 constructor가 `_mint(msg.sender, initialSupply)` 를 하므로 100만 개가 테스트 컨트랙트에 갑니다. 그다음 `transfer` 로 alice와 bob에게 1만 개씩 나눠줍니다. `transfer` 는 **호출한 쪽의** 토큰을 보내는 함수라서 "누구에게서"가 없습니다.

주소	SW42	R2B
테스트 컨트랙트 (배포자)	980,000	980,000
alice	10,000	10,000
bob	10,000	10,000
AMM 풀 (reserveA / reserveB)	0	0

3-2. 도우미: _addLiquidity

AMM.t.sol · 30–38

```
30      /// @dev who가 approve 2번 + addLiquidity 1번 (MetaMask 서명 3번과 같은 흐름)
31      function _addLiquidity(address who, uint256 amountA, uint256 amountB) internal returns (uint256)
{
32          vm.startPrank(who);
33          tokenA.approve(address(amm), amountA);
34          tokenB.approve(address(amm), amountB);
35          uint256 lp = amm.addLiquidity(amountA, amountB);
36          vm.stopPrank();
37          return lp;
38      }
```

MetaMask 서명 3번(SW42 approve, R2B approve, addLiquidity)을 한 함수로 묶은 것입니다. `who` 가 자기 토큰을 **풀에 넣는** 동작입니다. 받는 건 LP 토큰뿐입니다.

이름이 비슷한 두 함수를 구분하세요. 밑줄이 있는 `_addLiquidity` 는 **테스트 파일**의 포장지이고, 35줄에서 부르는 `amm.addLiquidity` 가 **AMM 컨트랙트**의 실제 함수입니다. reserve를 바꾸는 건 후자입니다.

prank 함정. 호출이 두 개 이상이면 반드시 `startPrank` 를 쓰세요. `vm.prank(bob)` 다음에 approve와 addLiquidity를 연달아 부르면, addLiquidity는 이미 prank가 끝나서 테스트 컨트랙트가 호출한 것이 됩니다.

3-3. test_FirstDeposit_MintsSqrt: 첫 공급

AMM.t.sol · 40–48

```
40      // ──────────── 노트 T1: 첫 공급 ────────────
41      function test_FirstDeposit_MintsSqrt() public {
42          uint256 lp = _addLiquidity(alice, 100 ether, 400 ether);
43
44          assertEq(lp, 200 ether); // √(100 × 400)
45          assertEq(amm.balanceOf(alice), 200 ether);
46          assertEq(amm.reserveA(), 100 ether);
47          assertEq(amm.reserveB(), 400 ether);
48      }
```

빈 풀에 alice가 SW42 100 + R2B 400을 넣습니다. `totalSupply() == 0` 이므로 첫 공급 공식을 씁니다.

> `LP = √(amountA × amountB) = √(100 × 400) = √40,000 =` **`200`**

줄	검사	왜 둘 다 확인하나
44	함수가 돌려준 값 `lp` = 200	함수가 "200개 줬다"고 **말한** 값
45	장부의 `amm.balanceOf(alice)` = 200	장부에 **실제로** 적힌 값. `_mint` 를 빼먹으면 44는 통과, 45는 실패
46-47	reserveA = 100, reserveB = 400	넣은 만큼 장부가 늘었나

잡는 버그: 첫 공급을 `sqrt(reserveA * reserveB)` 로 쓰면 빈 풀에서는 0이 되어 이 테스트가 실패합니다.

3-4. test_SecondDeposit_Proportional: 추가 공급

AMM.t.sol · 50–57

```
50      // ──────────── 노트 T2: 추가 공급 ────────────
51      function test_SecondDeposit_Proportional() public {
52          _addLiquidity(alice, 100 ether, 400 ether);
53          uint256 lp = _addLiquidity(bob, 100 ether, 400 ether);
54
55          assertEq(lp, 200 ether);
56          assertEq(amm.totalSupply(), 400 ether);
57      }
```

bob이 넣는 시점의 풀은 `100 / 400 / LP 200` 입니다. 추가 공급 공식은 이렇습니다.

> `LP = min(100 × 200 / 100, 400 × 200 / 400) = min(200, 200) =` **`200`**

`totalSupply()` 는 **LP 토큰**의 총량입니다. alice 200 + bob 200 = **400**. 풀의 R2B 양(800)과 헷갈리지 마세요. LP 개수와 풀 잔고는 다른 장부입니다.

3-5. test_RemoveLiquidity_Partial: 부분 출금

AMM.t.sol · 59–73

```
59      // ──────────── 노트 T4: 부분 출금 ────────────
60      function test_RemoveLiquidity_Partial() public {
61          _addLiquidity(alice, 100 ether, 400 ether);
```

```
62          _addLiquidity(bob, 100 ether, 400 ether);
63          // 풀: 200 / 800 / LP 400.  Bob이 LP 100개(25%)만 뺌
64
65          uint256 bobABefore = tokenA.balanceOf(bob);
66          vm.prank(bob); // 다음 호출 1번만 bob으로
67          (uint256 outA, uint256 outB) = amm.removeLiquidity(100 ether);
68
69          assertEq(outA, 50 ether); // 100 × 200 / 400
70          assertEq(outB, 200 ether); // 100 × 800 / 400
71          assertEq(tokenA.balanceOf(bob), bobABefore + outA); // 실제로 지갑에 들어왔나
72          assertEq(amm.balanceOf(bob), 100 ether); // 200 − 100
73      }
```

스왑 없이 공급 직후에 bob이 LP 400개 중 100개(25%)를 뺍니다. 줄마다 상태가 이렇게 바뀝니다.

시점	bob SW42	bob R2B	bob LP	풀 SW42	풀 R2B	LP 전체
setUp 직후	10,000	10,000	0	0	0	0
61 alice 공급	10,000	10,000	0	100	400	200
62 bob 공급	9,900	9,600	200	200	800	400
67 bob 출금	9,950	9,800	100	150	600	300

> `outA = 100 × 200 / 400 = 50` `outB = 100 × 800 / 400 = 200`

두 값이 왜 다른가. 같은 비율(25%)을 떼는데 더미 크기가 다르기 때문입니다. 피자 2판과 콜라 8병을 넷이 나누면 한 사람 몫은 피자 0.5판, 콜라 2병입니다. bob은 SW42 100 + R2B 400을 넣고 LP의 **절반**을 뺐으니 넣은 양의 **절반씩**(50, 200)을 돌려받습니다.

69-70줄은 함수가 **말한** 값, 71줄은 지갑에 **실제로** 들어왔는지, 72줄은 남은 LP입니다. 출금 양을 눈으로 보고 싶으면 `forge test --match-test test_RemoveLiquidity_Partial -vvvv` 의 트레이스에서 `← [Return] 5e19, 2e20` 을 찾으세요(5e19 = 50 ether).

3-6. test_RevertWhen_AddZero: 0을 넣으면 막혀야 한다

AMM.t.sol · 75–80

```
75      // ———————— 실패해야 하는 경우 ————————
76      function test_RevertWhen_AddZero() public {
77          vm.prank(alice);
78          vm.expectRevert(bytes("AMM: zero amount")); // addLiquidity의 require 메시지
79          amm.addLiquidity(0, 100 ether);
80      }
```

`expectRevert` 는 바로 다음 호출이 **이 메시지 그대로** revert해야 통과합니다. 컨트랙트의 `require` 문자열과 한 글자도 다르면 안 됩니다. revert될 호출이라 approve가 필요 없어서 도우미를 거치지 않고 `amm.addLiquidity` 를 직접 부릅니다.

3-7. 도우미: _swap

AMM.t.sol · 82–89

```
82      // ———————— Swap Test ————————
83      function _swap(address who, address tokenIn, uint256 amountIn, uint256 minOut) internal returns
(uint256) {
84          vm.startPrank(who);
85          IERC20(tokenIn).approve(address(amm), amountIn);
86          uint256 out = amm.swap(tokenIn, amountIn, minOut);
87          vm.stopPrank();
88          return out;
89      }
```

매개변수	뜻
`who`	스왑하는 사람
`tokenIn`	**내는(파는)** 토큰의 주소. 받는 토큰은 나머지 하나로 자동 결정
`amountIn`	내는 토큰의 양
`minOut`	**받는** 토큰의 최소 수령량. 0이면 검사 안 함

`tokenIn` 은 그냥 `address` 라서 `.approve()` 를 부를 수 없습니다. `IERC20(tokenIn)` 으로 감싸야 합니다. 스왑은 한 종류만 내므로 approve가 한 번뿐입니다.

3-8. test_Swap_AToB_Invariants: 정확한 값 대신 불변식

AMM.t.sol · 91–112

```
91      function test_Swap_AToB_Invariants() public {
92          _addLiquidity(alice, 100 ether, 400 ether);
93          _addLiquidity(bob, 100 ether, 400 ether);
94          // 풀: 200 / 800 (노트 T3 시작 상태)
95
96          uint256 kBefore = amm.reserveA() * amm.reserveB();
97          uint256 expected = amm.getAmountOut(50 ether, amm.reserveA(), amm.reserveB()); // 스왑 전 reserveA, reserveB
98
99          address carol = makeAddr("carol");
100         tokenA.transfer(carol, 50 ether);
101         uint256 amountOut = _swap(carol, address(tokenA), 50 ether, 0);
102
103         uint256 kAfter = amm.reserveA() * amm.reserveB();
104         uint256 noFeeOut = 800 ether * 50 ether / (200 ether + 50 ether); // 160 ether
105
106         assertGe(kAfter, kBefore, "k must not decrease"); // 규칙 ①
107         assertLt(amountOut, noFeeOut, "fee must reduce output"); // 규칙 ②
108         assertEq(amm.reserveA(), 250 ether, "reserveA grows by full amountIn"); // 규칙 ③
109         assertEq(amm.reserveB(), 800 ether - amountOut, "reserveB shrinks by amountOut"); // 규칙 ③
110         assertEq(amountOut, expected, "output matches getAmountOut preview"); // 규칙 ④
111         assertEq(tokenB.balanceOf(carol), amountOut, "carol received amountOut"); // carol 지갑에 실제로 들어왔나
112     }
```

carol이 SW42 50개를 넣으면 R2B를 **156.1368…개** 받습니다. 정수 나눗셈 때문에 끝자리까지 손으로 맞추기 어려워서, **어떤 값이 나오든 반드시 지켜져야 하는 규칙(불변식)**을 검사합니다.

	스왑 전	스왑 후 (수수료 3%)
reserveA	200	250 (+50, **전부** 들어옴)
reserveB	800	643.86 (−156.14)
k = A × B	160,000	160,965.8 (늘어남)

줄	규칙	왜
106	① `kAfter ≥ kBefore`	k가 줄면 풀이 받아야 할 것보다 더 내준 것. AMM의 가장 중요한 안전 규칙. 수수료가 0이면 같아도 정상이라 `Ge`
107	② `amountOut < noFeeOut` (160)	수수료가 실제로 적용됨. 같으면 수수료가 빠진 버그라서 `Lt`
108-109	③ 장부의 정확한 증감	reserveA는 50 **전부** 증가, reserveB는 준 양만큼 감소
110	④ 실제 = 미리보기	프론트가 `getAmountOut` 으로 보여준 값과 실제 결과가 같아야 함

왜 reserveA만 정확히 250인가. 수수료는 "덜 주는" 방식으로 걷습니다. carol이 낸 50개는 통째로 풀에 들어가고, carol에게 줄 양을 계산할 때만 48.5개로 칩니다. 그래서 넣은 쪽은 정확히 +50이고, 준 쪽만 수수료 없는 값(640)과 달라집니다. 계산에 쓰지 않은 1.5개가 풀에 남아 k를 키우고, 그게 LP의 수익입니다.

97줄은 반드시 스왑보다 위에. `expected` 는 스왑 **전** reserve로 미리 계산한 값입니다. 스왑 뒤에 계산하면 reserve가 이미 바뀌어 엉뚱한 기대값이 나옵니다. 이 변수를 만들어 놓고 쓰지 않으면 컴파일러가 `Unused local variable` 경고를 냅니다.

k > 0은 왜 부족한가. k가 160,000에서 100으로 줄어도 `k > 0` 은 통과합니다. 좋은 불변식은 정상일 때 통과하고, 잘못됐을 때 **반드시** 실패하는 규칙입니다.

3-9. test_RevertWhen_SlippageExceeded: 슬리피지 보호

AMM.t.sol · 114–123

```
114     function test_RevertWhen_SlippageExceeded() public {
115         _addLiquidity(alice, 100 ether, 400 ether);
116 
117         tokenA.transfer(bob, 50 ether);
118         vm.startPrank(bob);
119         tokenA.approve(address(amm), 50 ether);
120         vm.expectRevert(bytes("AMM: slippage")); // swap의 슬리피지 require 메시지
121         amm.swap(address(tokenA), 50 ether, 1_000 ether); // 1000개 이상 달라고 요구 → 불가능
122         vm.stopPrank();
123     }
```

풀은 alice만 공급한 `100 / 400` 입니다. bob이 SW42 50개를 내면 받을 수 있는 R2B는 약 130.6개인데, `minAmountOut` 으로 1,000개를 요구합니다. 불가능하므로 `"AMM: slippage"` 로 revert해야 합니다. 실제 서비스에서는 프론트가 예상 수령량의 99% 같은 값을 `minAmountOut` 으로 넣어서, 그사이에 가격이 크게 움직이면 거래가 취소되게 합니다.

4. NFTMarket.t.sol: NFT 마켓 테스트 8개

4-1. 준비: 상수, 이벤트 선언, setUp

NFTMarket.t.sol · 12–48

```
12 contract NFTMarketTest is Test {
13     AMM amm;
14     SunWoo42 tokenA;
15     TokenB tokenB;
16     TokenizeArt nft;
17 
```

```
    address alice = makeAddr("alice"); // 판매자
    address bob = makeAddr("bob"); // 구매자
    address carol = makeAddr("carol"); // 두 번째 구매자 (실패 케이스용)

    uint256 constant TOKEN_ID = 0; // alice가 받는 첫 NFT 번호 (TokenizeArt는 0번부터 발행)
    uint256 constant PRICE = 500 ether;

    // 이벤트를 expectEmit으로 비교하려면 테스트 쪽에도 같은 모양으로 선언해 둬야 합니다.
    event NFTListed(
        uint256 indexed listingId,
        address indexed seller,
        address indexed nft,
        uint256 tokenId,
        address paymentToken,
        uint256 price
    );

    /// @dev 토큰 2개 + AMM + NFT 배포, alice에게 NFT 0번, bob과 carol에게 결제용 토큰
    function setUp() public {
        tokenA = new SunWoo42(1_000_000 ether);
        tokenB = new TokenB(1_000_000 ether);
        amm = new AMM(address(tokenA), address(tokenB));
        nft = new TokenizeArt("ipfs://test-metadata"); // 이 테스트 컨트랙트가 owner → mint 가능

        nft.mint(alice); // alice가 NFT 0번을 가짐

        tokenA.transfer(bob, 10_000 ether);
        tokenB.transfer(bob, 10_000 ether);
        tokenA.transfer(carol, 10_000 ether);
        tokenB.transfer(carol, 10_000 ether);
    }
```

- **TokenizeArt를 배포한 테스트 컨트랙트가 owner**가 되므로 `nft.mint(alice)` 를 부를 수 있습니다. TokenizeArt는 0번부터 발행하므로 alice는 **0번** NFT를 받습니다(`TOKEN_ID = 0`).
- 판매자 alice는 토큰이 필요 없고, 구매자 bob과 carol에게만 결제용 토큰을 나눠줍니다.
- 26-33줄의 이벤트 선언은 `expectEmit` 때문에 필요합니다. 테스트 안에서 `emit NFTListed(...)` 로 "기대하는 이벤트"를 만들려면, 테스트 컨트랙트에도 **똑같은 모양**의 이벤트가 선언되어 있어야 합니다.

4-2. 도우미 세 개

NFTMarket.t.sol · 50–71

```
    // ──────────────────────────────── 도우미 ────────────────────────────────

    /// @dev alice가 approve 1번 + listNFT 1번 (MetaMask 서명 2번과 같은 흐름)
    function _listAsAlice(address paymentToken) internal returns (uint256 listingId) {
        vm.startPrank(alice);
        nft.approve(address(amm), TOKEN_ID);
        listingId = amm.listNFT(address(nft), TOKEN_ID, PRICE, paymentToken);
        vm.stopPrank();
    }

    /// @dev who가 결제 토큰 approve 1번 + buyNFT 1번
    function _buy(address who, address paymentToken, uint256 listingId) internal {
        vm.startPrank(who);
        IERC20(paymentToken).approve(address(amm), PRICE);
        amm.buyNFT(listingId);
        vm.stopPrank();
    }

    /// @dev listings(id)는 필드 6개를 튜플로 돌려줍니다. 그중 마지막(active)만 꺼냅니다.
    function _isActive(uint256 listingId) internal view returns (bool active) {
        (,,,,, active) = amm.listings(listingId);
```

```
71      }
```

도우미	하는 일	MetaMask로 치면
`_listAsAlice`	NFT approve → listNFT	서명 2번
`_buy`	결제 토큰 approve → buyNFT	서명 2번
`_isActive`	등록이 아직 판매 중인지 읽기	(읽기만, 가스 없음)

NFT도 approve가 필요합니다. `listNFT` 안에서 AMM이 `transferFrom(alice, AMM, 0)` 으로 NFT를 옮기기 때문입니다. ERC-721은 `approve(spender, tokenId)` 로 **그 NFT 하나만** 허락하거나, `setApprovalForAll` 로 전부 허락합니다.

구매자는 AMM에게 approve합니다. 대금은 alice에게 바로 가지만, `safeTransferFrom` 을 실행하는 "대리인"이 AMM이기 때문입니다.

70줄의 쉼표 문법. `public` mapping의 값이 struct이면, 자동 getter `amm.listings(id)` 는 필드 6개를 **튜플**로 돌려줍니다. 필요 없는 자리는 비워 두고 쉼표만 찍습니다.

```
(seller, nft, tokenId, price, paymentToken, active)
( , , , , , active) ← 마지막 하나만 받기
```

4-3. A. test_List_EscrowsNFTAndStoresListing: 등록

NFTMarket.t.sol · 73–87

```
73      // ──────────────────────────── A. 등록 ────────────────────────────
74
75      function test_List_EscrowsNFTAndStoresListing() public {
76          uint256 listingId = _listAsAlice(address(tokenA));
77
78          assertEq(listingId, 0, "first listing gets ID 0");
79          assertEq(amm.nextListingId(), 1, "next ID moves to 1");
80          assertEq(nft.ownerOf(TOKEN_ID), address(amm), "NFT is escrowed in AMM");
81
82          (address seller,, uint256 tokenId, uint256 price,, bool active) = amm.listings(listingId);
83          assertEq(seller, alice);
84          assertEq(tokenId, TOKEN_ID);
85          assertEq(price, PRICE);
86          assertTrue(active, "listing is active");
87      }
```

줄	확인	의미
78	첫 번호는 0	`listingId = nextListingId` 로 **지금** 번호에 저장했나. `listingId + 1` 로 쓰면 반환값이 항상 0에서 시작해 모든 등록이 1번 칸에 덮어써짐
79	다음 번호는 1	저장한 **다음에** 번호를 올렸나
80	NFT 주인 = AMM	**escrow**: 팔리거나 취소될 때까지 AMM이 보관
82-86	저장된 필드	판매자는 `msg.sender` (alice). `tx.origin` 을 쓰면 중간에 컨트랙트(멀티시그 등)가 끼었을 때 판매자가 엉뚱하게 기록됨

4-4. test_List_EmitsEvent: 이벤트 확인

NFTMarket.t.sol · 89–99

```
89      function test_List_EmitsEvent() public {
90          vm.startPrank(alice);
91          nft.approve(address(amm), TOKEN_ID); // approve를 먼저: 이것도 Approval 이벤트를 내기 때문
92
93          // 바로 다음 호출(listNFT)이 이 이벤트를 내야 함. (topic1, topic2, topic3, data) 모두 비교
94          vm.expectEmit(true, true, true, true, address(amm));
95          emit NFTListed(0, alice, address(nft), TOKEN_ID, address(tokenA), PRICE);
96
97          amm.listNFT(address(nft), TOKEN_ID, PRICE, address(tokenA));
98          vm.stopPrank();
99      }
```

서브그래프는 **이벤트만 보고** 데이터베이스를 만듭니다. 이벤트 필드가 틀리면 서브그래프 데이터도 틀립니다. 그래서 이벤트를 따로 검사합니다.

`expectEmit` 인자	비교 대상	NFTListed에서
1번째 true	topic1 (첫 indexed)	listingId
2번째 true	topic2	seller
3번째 true	topic3	nft
4번째 true	data (indexed 아닌 것)	tokenId, paymentToken, price
`address(amm)`	이벤트를 낸 컨트랙트	AMM이어야 함

실제로 겪은 함정. 처음에는 `expectEmit` 다음에 `_listAsAlice` 를 불렀습니다. 그런데 도우미 안에서 `nft.approve` 가 **먼저** 실행되어 `Approval` 이벤트가 나왔고, 테스트가 `Approval != expected NFTListed` 로 실패했습니다. `expectEmit` 은 **바로 다음 외부 호출**의 이벤트를 봅니다. 그래서 approve를 먼저 끝내고(91줄), 그다음에 `expectEmit` 을 걸고(94줄), 바로 `listNFT` 를 부릅니다(97줄).

4-5. B. test_Cancel_ReturnsNFTAndDeactivates: 취소

NFTMarket.t.sol · 101–111

```
101     // ──────────────────────────── B. 취소 ────────────────────────────
102
103     function test_Cancel_ReturnsNFTAndDeactivates() public {
104         uint256 listingId = _listAsAlice(address(tokenA));
105
106         vm.prank(alice);
107         amm.cancelListing(listingId);
108
109         assertEq(nft.ownerOf(TOKEN_ID), alice, "NFT is back with the seller");
110         assertFalse(_isActive(listingId), "listing must be inactive (storage, not memory)");
111     }
```

109줄은 NFT가 alice에게 돌아왔는지, 110줄은 **장부가 판매 종료로 바뀌었는지** 확인합니다. 110줄이 이 파일에서 가장 중요한 검사입니다.

	`Listing storage` (정답)	`Listing memory` (버그)
무엇	장부 **원본**을 가리킴	장부를 **복사**한 임시본
`active = false` 후 장부	false (취소됨)	**true 그대로** (복사본만 바뀌고 함수 끝나면 사라짐)
컴파일	성공	**성공** (에러 없음)
109줄	통과	통과 (NFT는 돌아옴)
110줄	통과	**실패**

정말 잡는지 확인했습니다. `cancelListing` 을 일부러 `memory` 로 바꿔서 돌리면 `[FAIL: listing must be inactive (storage, not memory)]` 가 나옵니다. 되돌리면 다시 통과합니다. 이렇게 일부러 버그를 넣어 테스트가 잡는지 보는 것을 **뮤테이션 테스트**라고 합니다.

memory 버그가 실제로 일으키는 사고: alice가 0번 등록(500)을 취소 → 장부는 여전히 판매 중 → alice가 같은 NFT를 900에 다시 등록(1번) → bob이 **옛 0번**을 500에 사서 NFT를 가져감. alice는 900에 팔려던 NFT를 500에 잃고, 1번 등록은 NFT 없이 판매 중으로 남습니다.

4-6. C. test_Buy_PaysSellerAndTransfersNFT: 구매

NFTMarket.t.sol · 113–128

```
113     // ──────────────────────────── C. 구매 ────────────────────────────
114
115     function test_Buy_PaysSellerAndTransfersNFT() public {
116         uint256 listingId = _listAsAlice(address(tokenB)); // 이번엔 R2B로 결제받는 등록
117
118         uint256 aliceBefore = tokenB.balanceOf(alice);
119         uint256 bobBefore = tokenB.balanceOf(bob);
120
121         _buy(bob, address(tokenB), listingId);
122
123         assertEq(nft.ownerOf(TOKEN_ID), bob, "buyer owns the NFT");
124         assertEq(tokenB.balanceOf(alice), aliceBefore + PRICE, "seller got paid");
125         assertEq(tokenB.balanceOf(bob), bobBefore - PRICE, "buyer paid");
126         assertEq(tokenB.balanceOf(address(amm)), 0, "payment never sits in AMM");
127         assertFalse(_isActive(listingId), "sold listing is inactive");
128     }
```

이번 등록은 **R2B(tokenB)**로 받습니다. A의 SW42 등록과 합쳐서 "Token A 또는 Token B로 결제"라는 서브젝트 조건을 둘 다 확인합니다.

시점	NFT 0번 주인	alice R2B	bob R2B	AMM R2B
등록 후	AMM	0	10,000	0
구매 후	bob	500	9,500	**0**

126줄은 설계 결정을 검사합니다. 대금은 AMM을 거치지 않고 **판매자에게 바로** 갑니다. AMM 금고에 NFT 대금이 섞이지 않으니 풀 장부(`reserveA/B`)와 완전히 분리됩니다. 127줄은 팔린 등록이 판매 종료로 바뀌었는지 봅니다.

대금 이동(`safeTransferFrom`)과 NFT 이동(`transferFrom`) 중 하나라도 실패하면 **트랜잭션 전체가 취소**됩니다. 돈만 내고 NFT를 못 받는 일은 생기지 않습니다(원자성).

4-7. D. test_RevertWhen_NonSellerCancels: 남이 취소 시도

NFTMarket.t.sol · 130–138

```
130     // ──────────────────────────────── D. 남이 취소 시도 ────────────────────────────────
131
132     function test_RevertWhen_NonSellerCancels() public {
133         uint256 listingId = _listAsAlice(address(tokenA));
134
135         vm.prank(bob);
136         vm.expectRevert(bytes("AMM: not the seller"));
137         amm.cancelListing(listingId);
138     }
```

`cancelListing`은 NFT를 `msg.sender`에게 돌려줍니다. 판매자 검사가 없으면 bob이 취소를 부르는 순간 **alice의 NFT가 bob에게** 갑니다. 이 테스트가 마켓의 가장 중요한 권한 검사를 지킵니다.

4-8. E. test_RevertWhen_BuyingSoldListing: 두 번 구매 시도

NFTMarket.t.sol · 140–151

```
140     // ──────────────────────────────── E. 두 번 구매 시도 ────────────────────────────────
141
142     function test_RevertWhen_BuyingSoldListing() public {
143         uint256 listingId = _listAsAlice(address(tokenA));
144         _buy(bob, address(tokenA), listingId);
145
146         vm.startPrank(carol);
147         tokenA.approve(address(amm), PRICE);
148         vm.expectRevert(bytes("AMM: listing not active"));
149         amm.buyNFT(listingId);
150         vm.stopPrank();
151     }
```

bob이 산 뒤 carol이 같은 등록을 사려고 합니다. `active`가 false이므로 `"AMM: listing not active"`로 막혀야 합니다. carol은 approve까지 정상적으로 했으므로, 실패 원인은 오직 `active` 검사입니다. 존재하지 않는 번호(예: 999)도 같은 이유로 막힙니다. mapping은 없는 키를 읽으면 모든 필드가 기본값(false, 0)인 struct를 돌려주기 때문입니다.

4-9. F. test_RevertWhen_InvalidPaymentToken: 잘못된 결제 토큰

NFTMarket.t.sol · 153–163

```
153     // ──────────────────────────────── F. 잘못된 결제 토큰 ────────────────────────────────
154
155     function test_RevertWhen_InvalidPaymentToken() public {
156         SunWoo42 otherToken = new SunWoo42(1 ether); // 풀과 무관한 토큰
157
158         vm.startPrank(alice);
159         nft.approve(address(amm), TOKEN_ID);
160         vm.expectRevert(bytes("AMM: invalid payment token"));
161         amm.listNFT(address(nft), TOKEN_ID, PRICE, address(otherToken));
162         vm.stopPrank();
163     }
```

풀과 무관한 토큰을 하나 새로 배포해서 결제 토큰으로 등록해 봅니다. 서브젝트는 대금을 **Token A 또는 Token B**로 받으라고 했으므로 막혀야 합니다. `swap`의 `tokenIn` 검사와 같은 비교입니다.

4-10. G. test_Marketplace_DoesNotTouchPoolReserves: 풀과 독립

NFTMarket.t.sol · 165–183

```
165     // ──────────────────────────────── G. 풀과 독립 ────────────────────────────────
```

```
166
167     function test_Marketplace_DoesNotTouchPoolReserves() public {
168         // 풀에 유동성을 먼저 넣어 둠 (bob이 공급자)
169         vm.startPrank(bob);
170         tokenA.approve(address(amm), 100 ether);
171         tokenB.approve(address(amm), 400 ether);
172         amm.addLiquidity(100 ether, 400 ether);
173         vm.stopPrank();
174
175         uint256 reserveABefore = amm.reserveA();
176         uint256 reserveBBefore = amm.reserveB();
177
178         uint256 listingId = _listAsAlice(address(tokenA));
179         _buy(carol, address(tokenA), listingId);
180
181         assertEq(amm.reserveA(), reserveABefore, "NFT sale must not change reserveA");
182         assertEq(amm.reserveB(), reserveBBefore, "NFT sale must not change reserveB");
183     }
```

서브젝트 문구: *"이 기능은 상수곱 공식을 쓰지 않고 풀 로직과 완전히 독립적이다."* 이 문장을 그대로 검사로 바꾼 테스트입니다. 풀에 유동성(100 / 400)을 넣어 두고, NFT를 SW42로 사고팔아도 `reserveA`, `reserveB` 가 **전혀 변하지 않아야** 합니다.

5. 실패 메시지 읽는 법

Foundry는 실패 메시지 가운데에 **"실제로는 이랬다"**는 기호를 넣습니다. 그 기호로 어떤 assert가 실패했는지 알 수 있습니다.

실패한 함수	검사한 것	실패 메시지의 기호
`assertEq(a, b)`	a == b	`a != b`
`assertGe(a, b)`	a ≥ b	`a < b`
`assertLt(a, b)`	a < b	`a >= b`
`expectRevert`	revert 메시지	`Error != expected error: 실제 != 기대`
`expectEmit`	이벤트	`실제이벤트 != expected 기대이벤트`

```
[FAIL: assertion failed: 15613682092555331991 9 < 16000000000000000000]
156.1368… ether (실제) < 160 ether (기대) → assertGe가 실패
```

- **숫자 읽기:** 끝 18자리를 떼면 "개" 단위입니다. `50000000000000000000` = 50 ether = 50개.
- **첫 실패에서 멈춤:** 한 테스트 안에서 assert가 실패하면 그 아래 줄은 실행되지 않습니다. 하나 고치면 다음 줄에서 또 실패할 수 있습니다.
- **메시지 인자를 붙이세요:** `assertEq(a, b, "reserveB shrinks by amountOut")` 처럼 쓰면 기호를 해석할 필요 없이 어느 줄인지 바로 보입니다. 메시지는 **그 줄이 실제로 검사하는 내용**과 맞아야 합니다.
- **테스트가 틀렸을 수도 있다:** 실패하면 먼저 손으로 계산해 보세요. 실제 값이 내 계산과 같으면 기대값(테스트)을 고치고, 다르면 컨트랙트를 의심합니다.
- **저장했나:** `forge test` 는 디스크의 파일만 읽습니다. 고친 뒤 저장하지 않으면 예전 결과가 나옵니다.

6. 서브젝트 요구사항과 테스트 대응표

서브젝트 요구사항	증명하는 테스트
유동성 추가와 LP 발행	FirstDeposit_MintsSqrt, SecondDeposit_Proportional
전액·부분 출금	RemoveLiquidity_Partial
상수곱 공식 스왑	Swap_AToB_Invariants (규칙 ①③④)
수수료 1~10%	Swap_AToB_Invariants (규칙 ②)
잘못된 입력 차단	RevertWhen_AddZero
슬리피지 보호 (보너스)	RevertWhen_SlippageExceeded
NFT 등록 + escrow	List_EscrowsNFTAndStoresListing
판매자 취소	Cancel_ReturnsNFTAndDeactivates, RevertWhen_NonSellerCancels
Token A 또는 B로 구매	List_*(SW42), Buy_PaysSellerAndTransfersNFT(R2B), RevertWhen_InvalidPaymentToken
재판매·중복 구매 방지	RevertWhen_BuyingSoldListing
풀 로직과 독립	Marketplace_DoesNotTouchPoolReserves, Buy(126줄)
서브그래프용 이벤트	List_EmitsEvent

아직 테스트하지 않은 것: B → A 방향 스왑, `LiquidityAdded` · `Swap` · `NFTSold` 등 나머지 이벤트의 필드, 비율이 어긋난 공급(min 쪽 기준). 서브그래프를 만들기 전에 이벤트 테스트를 더 추가하면 좋습니다.

7. 부록: 테스트 파일 전체

code/test/AMM.t.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {Test} from "forge-std/Test.sol";
import {AMM} from "../src/AMM.sol";
import {SunWoo42} from "../src/SunWoo42.sol";
import {TokenB} from "../src/tokenB.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

contract AMMTest is Test {
    AMM amm;
    SunWoo42 tokenA;
    TokenB tokenB;

    address alice = makeAddr("alice");
    address bob = makeAddr("bob");

    /// @dev 각 테스트 전에 실행: 토큰 2개 + AMM 배포, alice/bob에게 토큰 나눠주기
    function setUp() public {
        tokenA = new SunWoo42(1_000_000 ether); // 이 테스트 컨트랙트가 배포자 → 전량 받음
        tokenB = new TokenB(1_000_000 ether);
        amm = new AMM(address(tokenA), address(tokenB));

        tokenA.transfer(alice, 10_000 ether);
        tokenB.transfer(alice, 10_000 ether);
        tokenA.transfer(bob, 10_000 ether);
        tokenB.transfer(bob, 10_000 ether);
    }

    /// @dev who가 approve 2번 + addLiquidity 1번 (MetaMask 서명 3번과 같은 흐름)
    function _addLiquidity(address who, uint256 amountA, uint256 amountB) internal returns (uint256)
{
        vm.startPrank(who);
        tokenA.approve(address(amm), amountA);
        tokenB.approve(address(amm), amountB);
        uint256 lp = amm.addLiquidity(amountA, amountB);
        vm.stopPrank();
        return lp;
    }

    // ──────────── 노트 T1: 첫 공급 ────────────
    function test_FirstDeposit_MintsSqrt() public {
        uint256 lp = _addLiquidity(alice, 100 ether, 400 ether);

        assertEq(lp, 200 ether); // √(100 × 400)
        assertEq(amm.balanceOf(alice), 200 ether);
        assertEq(amm.reserveA(), 100 ether);
        assertEq(amm.reserveB(), 400 ether);
    }

    // ──────────── 노트 T2: 추가 공급 ────────────
    function test_SecondDeposit_Proportional() public {
        _addLiquidity(alice, 100 ether, 400 ether);
        uint256 lp = _addLiquidity(bob, 100 ether, 400 ether);

        assertEq(lp, 200 ether);
        assertEq(amm.totalSupply(), 400 ether);
    }

```

```
    // ──────────── 노트 T4: 부분 출금 ────────────
    function test_RemoveLiquidity_Partial() public {
        _addLiquidity(alice, 100 ether, 400 ether);
        _addLiquidity(bob, 100 ether, 400 ether);
        // 풀: 200 / 800 / LP 400.  Bob이 LP 100개(25%)만 뺌

        uint256 bobABefore = tokenA.balanceOf(bob);
        vm.prank(bob); // 다음 호출 1번만 bob으로
        (uint256 outA, uint256 outB) = amm.removeLiquidity(100 ether);

        assertEq(outA, 50 ether); // 100 × 200 / 400
        assertEq(outB, 200 ether); // 100 × 800 / 400
        assertEq(tokenA.balanceOf(bob), bobABefore + outA); // 실제로 지갑에 들어왔나
        assertEq(amm.balanceOf(bob), 100 ether); // 200 - 100
    }

    // ──────────── 실패해야 하는 경우 ────────────
    function test_RevertWhen_AddZero() public {
        vm.prank(alice);
        vm.expectRevert(bytes("AMM: zero amount")); // addLiquidity의 require 메시지
        amm.addLiquidity(0, 100 ether);
    }

    // ──────────── Swap Test ────────────
    function _swap(address who, address tokenIn, uint256 amountIn, uint256 minOut) internal returns
(uint256) {
        vm.startPrank(who);
        IERC20(tokenIn).approve(address(amm), amountIn);
        uint256 out = amm.swap(tokenIn, amountIn, minOut);
        vm.stopPrank();
        return out;
    }

    function test_Swap_AToB_Invariants() public {
        _addLiquidity(alice, 100 ether, 400 ether);
        _addLiquidity(bob, 100 ether, 400 ether);
        // 풀: 200 / 800 (노트 T3 시작 상태)

        uint256 kBefore = amm.reserveA() * amm.reserveB();
        uint256 expected = amm.getAmountOut(50 ether, amm.reserveA(), amm.reserveB()); // 스왑 전
reserveA, reserveB

        address carol = makeAddr("carol");
        tokenA.transfer(carol, 50 ether);
        uint256 amountOut = _swap(carol, address(tokenA), 50 ether, 0);

        uint256 kAfter = amm.reserveA() * amm.reserveB();
        uint256 noFeeOut = 800 ether * 50 ether / (200 ether + 50 ether); // 160 ether

        assertGe(kAfter, kBefore, "k must not decrease"); // 규칙 ①
        assertLt(amountOut, noFeeOut, "fee must reduce output"); // 규칙 ②
        assertEq(amm.reserveA(), 250 ether, "reserveA grows by full amountIn"); // 규칙 ③
        assertEq(amm.reserveB(), 800 ether - amountOut, "reserveB shrinks by amountOut"); // 규칙 ③
        assertEq(amountOut, expected, "output matches getAmountOut preview"); // 규칙 ④
        assertEq(tokenB.balanceOf(carol), amountOut, "carol received amountOut"); // carol 지갑에 실제로
들어왔나
    }

    function test_RevertWhen_SlippageExceeded() public {
        _addLiquidity(alice, 100 ether, 400 ether);

        tokenA.transfer(bob, 50 ether);
        vm.startPrank(bob);
        tokenA.approve(address(amm), 50 ether);
        vm.expectRevert(bytes("AMM: slippage")); // swap의 슬리피지 require 메시지
        amm.swap(address(tokenA), 50 ether, 1_000 ether); // 1000개 이상 달라고 요구 → 불가능
```

```
122         vm.stopPrank();
123     }
124 }
```

code/test/NFTMarket.t.sol

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {Test} from "forge-std/Test.sol";
import {AMM} from "../src/AMM.sol";
import {SunWoo42} from "../src/SunWoo42.sol";
import {TokenB} from "../src/tokenB.sol";
import {TokenizeArt} from "../src/TokenizeArt.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

/// @dev Tests for the fixed-price NFT marketplace inside AMM (listNFT / cancelListing / buyNFT).
contract NFTMarketTest is Test {
    AMM amm;
    SunWoo42 tokenA;
    TokenB tokenB;
    TokenizeArt nft;

    address alice = makeAddr("alice"); // 판매자
    address bob = makeAddr("bob"); // 구매자
    address carol = makeAddr("carol"); // 두 번째 구매자 (실패 케이스용)

    uint256 constant TOKEN_ID = 0; // alice가 받는 첫 NFT 번호 (TokenizeArt는 0번부터 발행)
    uint256 constant PRICE = 500 ether;

    // 이벤트를 expectEmit으로 비교하려면 테스트 쪽에도 같은 모양으로 선언해 둬야 합니다.
    event NFTListed(
        uint256 indexed listingId,
        address indexed seller,
        address indexed nft,
        uint256 tokenId,
        address paymentToken,
        uint256 price
    );

    /// @dev 토큰 2개 + AMM + NFT 배포, alice에게 NFT 0번, bob과 carol에게 결제용 토큰
    function setUp() public {
        tokenA = new SunWoo42(1_000_000 ether);
        tokenB = new TokenB(1_000_000 ether);
        amm = new AMM(address(tokenA), address(tokenB));
        nft = new TokenizeArt("ipfs://test-metadata"); // 이 테스트 컨트랙트가 owner → mint 가능

        nft.mint(alice); // alice가 NFT 0번을 가짐

        tokenA.transfer(bob, 10_000 ether);
        tokenB.transfer(bob, 10_000 ether);
        tokenA.transfer(carol, 10_000 ether);
        tokenB.transfer(carol, 10_000 ether);
    }

    // ──────────────────────────────── 도우미 ────────────────────────────────

    /// @dev alice가 approve 1번 + listNFT 1번 (MetaMask 서명 2번과 같은 흐름)
    function _listAsAlice(address paymentToken) internal returns (uint256 listingId) {
        vm.startPrank(alice);
        nft.approve(address(amm), TOKEN_ID);
        listingId = amm.listNFT(address(nft), TOKEN_ID, PRICE, paymentToken);
        vm.stopPrank();
    }

    /// @dev who가 결제 토큰 approve 1번 + buyNFT 1번
    function _buy(address who, address paymentToken, uint256 listingId) internal {
        vm.startPrank(who);
        IERC20(paymentToken).approve(address(amm), PRICE);
        amm.buyNFT(listingId);
```

```
        vm.stopPrank();
    }

    /// @dev listings(id)는 필드 6개를 튜플로 돌려줍니다. 그중 마지막(active)만 꺼냅니다.
    function _isActive(uint256 listingId) internal view returns (bool active) {
        (,,,,, active) = amm.listings(listingId);
    }

    // ──────────────────────────────── A. 등록 ────────────────────────────────

    function test_List_EscrowsNFTAndStoresListing() public {
        uint256 listingId = _listAsAlice(address(tokenA));

        assertEq(listingId, 0, "first listing gets ID 0");
        assertEq(amm.nextListingId(), 1, "next ID moves to 1");
        assertEq(nft.ownerOf(TOKEN_ID), address(amm), "NFT is escrowed in AMM");

        (address seller,, uint256 tokenId, uint256 price,, bool active) = amm.listings(listingId);
        assertEq(seller, alice);
        assertEq(tokenId, TOKEN_ID);
        assertEq(price, PRICE);
        assertTrue(active, "listing is active");
    }

    function test_List_EmitsEvent() public {
        vm.startPrank(alice);
        nft.approve(address(amm), TOKEN_ID); // approve를 먼저: 이것도 Approval 이벤트를 내기 때문

        // 바로 다음 호출(listNFT)이 이 이벤트를 내야 함. (topic1, topic2, topic3, data) 모두 비교
        vm.expectEmit(true, true, true, true, address(amm));
        emit NFTListed(0, alice, address(nft), TOKEN_ID, address(tokenA), PRICE);

        amm.listNFT(address(nft), TOKEN_ID, PRICE, address(tokenA));
        vm.stopPrank();
    }

    // ──────────────────────────────── B. 취소 ────────────────────────────────

    function test_Cancel_ReturnsNFTAndDeactivates() public {
        uint256 listingId = _listAsAlice(address(tokenA));

        vm.prank(alice);
        amm.cancelListing(listingId);

        assertEq(nft.ownerOf(TOKEN_ID), alice, "NFT is back with the seller");
        assertFalse(_isActive(listingId), "listing must be inactive (storage, not memory)");
    }

    // ──────────────────────────────── C. 구매 ────────────────────────────────

    function test_Buy_PaysSellerAndTransfersNFT() public {
        uint256 listingId = _listAsAlice(address(tokenB)); // 이번엔 R2B로 결제받는 등록

        uint256 aliceBefore = tokenB.balanceOf(alice);
        uint256 bobBefore = tokenB.balanceOf(bob);

        _buy(bob, address(tokenB), listingId);

        assertEq(nft.ownerOf(TOKEN_ID), bob, "buyer owns the NFT");
        assertEq(tokenB.balanceOf(alice), aliceBefore + PRICE, "seller got paid");
        assertEq(tokenB.balanceOf(bob), bobBefore - PRICE, "buyer paid");
        assertEq(tokenB.balanceOf(address(amm)), 0, "payment never sits in AMM");
        assertFalse(_isActive(listingId), "sold listing is inactive");
    }

    // ──────────────────────────────── D. 남이 취소 시도 ────────────────────────────────
```

```
131
132     function test_RevertWhen_NonSellerCancels() public {
133         uint256 listingId = _listAsAlice(address(tokenA));
134
135         vm.prank(bob);
136         vm.expectRevert(bytes("AMM: not the seller"));
137         amm.cancelListing(listingId);
138     }
139
140     // ──────────────────────────────── E. 두 번 구매 시도 ────────────────────────────────
141
142     function test_RevertWhen_BuyingSoldListing() public {
143         uint256 listingId = _listAsAlice(address(tokenA));
144         _buy(bob, address(tokenA), listingId);
145
146         vm.startPrank(carol);
147         tokenA.approve(address(amm), PRICE);
148         vm.expectRevert(bytes("AMM: listing not active"));
149         amm.buyNFT(listingId);
150         vm.stopPrank();
151     }
152
153     // ──────────────────────────────── F. 잘못된 결제 토큰 ────────────────────────────────
154
155     function test_RevertWhen_InvalidPaymentToken() public {
156         SunWoo42 otherToken = new SunWoo42(1 ether); // 풀과 무관한 토큰
157
158         vm.startPrank(alice);
159         nft.approve(address(amm), TOKEN_ID);
160         vm.expectRevert(bytes("AMM: invalid payment token"));
161         amm.listNFT(address(nft), TOKEN_ID, PRICE, address(otherToken));
162         vm.stopPrank();
163     }
164
165     // ──────────────────────────────── G. 풀과 독립 ────────────────────────────────
166
167     function test_Marketplace_DoesNotTouchPoolReserves() public {
168         // 풀에 유동성을 먼저 넣어 둠 (bob이 공급자)
169         vm.startPrank(bob);
170         tokenA.approve(address(amm), 100 ether);
171         tokenB.approve(address(amm), 400 ether);
172         amm.addLiquidity(100 ether, 400 ether);
173         vm.stopPrank();
174
175         uint256 reserveABefore = amm.reserveA();
176         uint256 reserveBBefore = amm.reserveB();
177
178         uint256 listingId = _listAsAlice(address(tokenA));
179         _buy(carol, address(tokenA), listingId);
180
181         assertEq(amm.reserveA(), reserveABefore, "NFT sale must not change reserveA");
182         assertEq(amm.reserveB(), reserveBBefore, "NFT sale must not change reserveB");
183     }
184 }
```